

Guía de Implementación — MVP Bajoneá

Versión: 1.0 **Fecha:** Julio 2026 **Alcance:** Desde `spring init` hasta MVP funcional con testing E2E (Playwright) y testing de API (Postman)

Cómo usar esta guía

Esta guía está pensada para ejecutarse en **Claude Code**, como una secuencia de fases. Cada fase indica:

- **Qué configurar vos** (skills, agentes, hooks, estructura) antes de pedirle a Claude Code que ejecute.
- **Qué pedirle a Claude Code** una vez que la configuración está lista.
- **Qué validar** antes de pasar a la fase siguiente.

Siguiendo el PDF de buenas prácticas: la **memoria de proyecto** (`CLAUDE.md`) se define una sola vez al principio y se actualiza en cada fase; las **skills** se crean para tareas repetibles (generar entidad+DTO+repo+service+controller, generar test Postman, generar test Playwright); los **agentes** se reservan para tareas acotadas y delegables (auditor de capa, generador de colección Postman); los **hooks** se usan para lo que debe cumplirse siempre sin excepción (formateo, evitar exponer entidades en controllers, bloquear escritura fuera de `src/`).

No se arma un plugin todavía: es una única persona (vos) en un único proyecto. El plugin tendría sentido si esto se fuera a distribuir a un equipo — no es el caso.

Índice

- 0. Decisiones cerradas de alcance del MVP
- 1. Fase 0 — Preparación del entorno de trabajo con Claude Code
- 2. Fase 1 — Inicialización del proyecto Spring Boot
- 3. Fase 2 — Base de datos: modelo MVP, Flyway y phpMyAdmin
- 4. Fase 2bis — Carga del catálogo geográfico (ETL Georef)
- 5. Fase 3 — Estructura de capas y convenciones de código
- 6. Fase 4 — Entities (JPA)
- 7. Fase 5 — DTOs y mapeo
- 8. Fase 6 — Repositories
- 9. Fase 7 — Configuración: seguridad, JWT y autorización por rol
- 10. Fase 8 — Services (lógica de negocio)
- 11. Fase 9 — Controllers y contrato de respuestas HTTP
- 12. Fase 10 — Email real (SMTP) para verificación de cuenta

13. Fase 11 — Cloudinary: subida firmada de imágenes de producto
14. Fase 12 — Notificaciones in-app (polling)
15. Fase 13 — Documentación automática (springdoc-openapi)
16. Fase 14 — Testing de API con Postman
17. Fase 15 — Figma: checklist de pantallas MVP
18. Fase 16 — Frontend HTML/CSS/JS
19. Fase 17 — Testing E2E con Playwright
20. Fase 18 — Integración final y checklist de cierre
21. Anexo A — Skills a crear (detalle completo)
22. Anexo B — Agentes a crear (detalle completo)
23. Anexo C — Hooks a crear (detalle completo)
24. Anexo D — Estructura de carpetas final del repo

0. Decisiones cerradas de alcance del MVP

Estas decisiones fueron confirmadas antes de escribir la guía y son vinculantes para todas las fases siguientes. Si en el camino aparece la tentación de "agregar solo esto que ya está documentado en el proyecto completo", volver a este punto 0 primero.

Tema	Decisión MVP
Carrito	Simplificado: 1 registro por cliente, 1 comercio a la vez, agregar/quitar/ver/vaciar. Sin expiración, sin timeouts, sin inactivación automática.
Pago / MercadoPago	Fuera del MVP. El pedido nace directo en estado <code>PENDIENTE</code> (no existe <code>PENDIENTE_PAGO</code>). No se crea entidad <code>Pago</code> ni <code>NotaCredito</code> ni <code>CuentaMercadoPago</code> en esta etapa.
Notificaciones	Solo in-app vía polling REST . Sin email para notificaciones de pedidos. Tabla <code>Notificacion</code> básica (usuario, mensaje, leído, fecha).
Verificación de cuenta	Token real + email real vía SMTP. Se genera un <code>Token</code> tipo <code>VERIFICACION_EMAIL</code> , se envía por email real, y hay un endpoint que lo valida y activa al usuario. Esto obliga a resolver el proveedor SMTP ya en el MVP (ver Fase 10).
Autenticación	JWT (jjwt) con rol embebido en el token. Sin sesiones server-side.
Figma	Se define primero un checklist cerrado de pantallas MVP (Fase 15); recién con ese checklist aprobado se diseña en Figma. No se tocan las ~100 pantallas del proyecto completo.
Roles y flujos incluidos	Registro Cliente, Registro Comercio, Login (3 roles), Logout (3 roles), Aprobación/Rechazo de Comercio (admin), CRUD Producto (comercio) con imágenes vía Cloudinary, CRUD Categoría y Tag (admin), Catálogo público de comercios, Carrito, Pedido simulado (POST → notifica a comercio → comercio acepta/rechaza → notifica a cliente).
Direcciones	Modelo completo, no simplificado. El registro de Cliente y de Comercio pide dirección con selector de Provincia → Localidad (dependiente), igual que en el proyecto completo. Esto requiere el catálogo <code>Provincia / Localidad</code> precargado por ETL desde la API Georef antes de poder registrar el primer usuario (ver Fase 2bis).
Imágenes de producto	Cloudinary + entidad <code>ImagenProducto</code> real , igual que en el proyecto completo: hasta 5 imágenes por producto, subida firmada desde el frontend, la base de datos guarda cada URL en su propia fila (<code>producto_id</code> , <code>url</code> , <code>orden</code> , <code>es_principal</code>), el backend valida el límite de 5 antes de firmar la subida. No se simplifica a una sola columna — el MVP contempla la galería completa porque el CRUD de productos la necesita para ser representativo del alcance real.

Tema	Decisión MVP
Explícitamente fuera de alcance	MercadoPago, WhatsApp (<code>wa.me</code> o integración), reclamos, soporte, reembolsos, jobs de expiración/timeout, suspensión de comercio/cliente, historial de estados, múltiples direcciones de cliente (una sola dirección al registrarse, ampliable después), horarios de atención granulares (se puede modelar como campo simple si hace falta mostrar algo, pero no bloquea pedidos por horario), galería de imágenes múltiple por producto (ver fila anterior).
Puerto	8080 , configurado explícitamente en <code>application.properties</code> para que no haya sorpresas si el entorno tiene otra convención.

Entidades del modelo completo que SÍ entran al MVP

`Provincia` , `Localidad` (catálogo estático, ver Fase 2bis), `Usuario` (con discriminación de rol simplificada — ver Fase 4), `PersonaFisica` , `PersonaJuridica` , `Cliente` , `Comercio` , `Administrador` , `Direccion` (modelo completo: calle, número, piso/depto, código postal, `localidad_id`), `Token` (solo tipo `VERIFICACION_EMAIL`), `Categoria` , `Tag` , `Producto` , `ImagenProducto` (galería completa, hasta 5 imágenes por producto, vía Cloudinary), `ProductoTag` , `Carrito` , `ItemCarrito` , `Pedido` , `DetallePedido` , `Notificacion` .

Entidades que NO entran al MVP (quedan documentadas pero no se implementan)

`HistorialEstadoUsuario` , `HistorialEstadoComercio` , `HistorialEstadoPedido` , `Sesion` , `Soporte` , `Reclamo` , `ConfiguracionTarifa` , `CuentaMercadoPago` , `Pago` , `NotaCredito` .

Fase 0 — Preparación del entorno de trabajo con Claude Code

Esta fase es 100% tuya, antes de escribir una sola línea de código.

0.1 Crear la estructura de carpetas de configuración del asistente

```
bajonea/
├─ CLAUDE.md                <- memoria principal (crear en 0.2)
├─ .claude/
│   ├─ skills/
│   ├─ agents/
│   └─ hooks/
├─ docs/
│   └─ DECISIONES.md        <- crear en 0.3
├─ backend/                 <- proyecto Spring Boot (Fase 1)
├─ frontend/                <- HTML/CSS/JS (Fase 16)
├─ postman/                 <- colecciones (Fase 14)
└─ testing/
    └─ playwright/          <- specs E2E (Fase 17)
```

Qué hacer vos: crear esta estructura de carpetas vacía en tu máquina o pedirle a Claude Code que la genere en la primera sesión.

0.2 Crear `CLAUDE.md` (memoria principal del proyecto)

Según el PDF: 300-500 líneas, se envía en cada turno, no debe contener el detalle histórico de decisiones (eso va en `docs/DECISIONES.md`).

Contenido mínimo a incluir:

- Descripción de una línea del proyecto (Bajoneá — plataforma de pedidos gastronómicos, TFC).
- Stack: Java 21, Spring Boot 3.x, MySQL, Flyway, JWT, HTML/CSS/JS vanilla.
- Puerto 8080.
- Estructura de paquetes del backend (la de la Fase 3).
- Reglas transversales no negociables (ver 0.4).
- Referencia a `docs/DECISIONES.md` para el detalle de por qué se tomó cada decisión.
- Referencia a este archivo de guía como fuente de la secuencia de fases.

Qué pedirle a Claude Code: "Leé esta guía de implementación y generá un `CLAUDE.md` inicial de no más de 400 líneas con lo que ya sabemos del proyecto hasta la Fase 0."

0.3 Crear `docs/DECISIONES.md`

Arranca vacío con un encabezado. Cada vez que se cierre una decisión de diseño durante la implementación (ej. "el carrito se identifica por `cliente_id` único"), se anota ahí con fecha y una línea de justificación. `CLAUDE.md` solo referencia este archivo, no repite su contenido.

0.4 Reglas transversales no negociables (van en `CLAUDE.md`)

Estas son las reglas que vos definiste y que deben aplicarse siempre, sin que dependan de que Claude Code "se acuerde" en cada prompt. Varias de ellas se refuerzan además con **hooks** (Anexo C), que es el mecanismo correcto para lo que debe cumplirse siempre según el PDF (sección 6, "Skill enseña el cómo, hook impone la regla").

1. Usar `List<T>` como colección por defecto en toda la capa de servicios y repositorios. Solo usar `Set<T>` cuando haya una razón concreta (ej. evitar duplicados en una relación M:N como `ProductoTag`) y `Map` cuando el acceso sea por clave. Justificar en el código con un comentario breve cualquier excepción.
2. Los controllers **nunca** devuelven una entidad JPA directamente. Siempre devuelven un DTO de respuesta (`XxxResponseDTO`) envuelto en `ResponseEntity<?>`.
3. Toda respuesta HTTP de mutación (POST/PUT/PATCH/DELETE) devuelve un cuerpo con al menos un campo `mensaje` (o una clase `ApiResponse` estándar — ver Fase 9), además del recurso si corresponde.
4. Códigos de estado HTTP explícitos y semánticos, nunca `200` por defecto para todo:
 - `201 Created` al crear (registro, producto, categoría, pedido).
 - `200 OK` para lecturas y actualizaciones.
 - `204 No Content` para eliminaciones sin cuerpo de respuesta (evaluar si conviene devolver mensaje igual; en este proyecto se prioriza el mensaje, ver Fase 9).

- `400 Bad Request` para validaciones de entrada fallidas.
- `401 Unauthorized` para token ausente/inválido.
- `403 Forbidden` para rol sin permiso.
- `404 Not Found` para recurso inexistente.
- `409 Conflict` para violaciones de estado/regla de negocio (ej. email duplicado, producto de otro comercio en el carrito).

5. Nunca hardcodear el puerto en ningún lado más que `application.properties`.

6. Nunca commitear credenciales (contraseña de MySQL, SMTP, `jwt.secret`) — siempre vía variables de entorno o `application-local.properties` ignorado por git.

Fase 1 — Inicialización del proyecto Spring Boot

1.1 Generar el proyecto base

Qué pedirle a Claude Code:

"Iniciá un proyecto Spring Boot 3.x con Java 21 y Maven en `backend/`, con estas dependencias: Spring Web, Spring Data JPA, MySQL Driver, Spring Security, Validation, Lombok, springdoc-openapi-starter-webmvc-ui, Spring Mail, flyway-core, flyway-mysql. Agregá también jjwt (api, impl, jackson) como dependencias manuales en el `pom.xml` ya que no están en Spring Initializr."

Esto puede hacerse con `curl` contra `start.spring.io` o editando el `pom.xml` a mano — Claude Code decide según lo que tenga disponible en el entorno.

1.2 Verificar `pom.xml`

Validar que incluya (nombres de artefacto de referencia, la versión exacta la resuelve Claude Code contra Maven Central):

```
spring-boot-starter-web
spring-boot-starter-data-jpa
spring-boot-starter-security
spring-boot-starter-validation
spring-boot-starter-mail
mysql-connector-j
org.flywaydb:flyway-core
org.flywaydb:flyway-mysql
org.projectlombok:lombok
org.springdoc:springdoc-openapi-starter-webmvc-ui
io.jsonwebtoken:jjwt-api
io.jsonwebtoken:jjwt-impl
io.jsonwebtoken:jjwt-jackson
```

1.3 `application.properties` inicial

```
server.port=8080

spring.datasource.url=jdbc:mysql://localhost:3306/bajonea
spring.datasource.username=${DB_USER:root}
spring.datasource.password=${DB_PASSWORD:}

spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

spring.flyway.enabled=true
spring.flyway.locations=classpath:db/migration

jwt.secret=${JWT_SECRET:cambiar-en-produccion-por-variable-de-entorno}
jwt.expiration-ms=86400000

spring.mail.host=${SMTP_HOST}
spring.mail.port=${SMTP_PORT:587}
spring.mail.username=${SMTP_USER}
spring.mail.password=${SMTP_PASSWORD}
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
```

Nota clave: `ddl-auto=validate`, no `update` ni `create`. El esquema lo gestiona **exclusivamente Flyway**. Esto evita que Hibernate "invente" columnas y que el modelo de datos se desincronice de las migraciones versionadas — coherente con el principio de consistencia que ya venís aplicando en toda la documentación.

1.4 Validar que levanta

Qué pedirle a Claude Code: que corra `mvn spring-boot:run` (o el wrapper `./mvnw`) y confirme que arranca en el puerto 8080 sin base de datos todavía (va a fallar en el intento de conexión, es esperado — se resuelve en la Fase 2). Lo importante acá es que compile y que el `pom.xml` esté sano antes de seguir.

Checklist de cierre de Fase 1

- `[] mvn clean compile` sin errores.
 - `[]` Estructura estándar Maven (`src/main/java`, `src/main/resources`, `src/test/java`).
 - `[] pom.xml` con todas las dependencias de 1.2.
 - `[] application.properties` con placeholders de variables de entorno, sin credenciales reales.
-

Fase 2 — Base de datos: modelo MVP, Flyway y phpMyAdmin

2.1 Instalar/configurar MySQL + phpMyAdmin localmente

Esto es un paso tuyo (fuera de Claude Code, es infraestructura local): instalar MySQL Server y phpMyAdmin (vía XAMPP, Laragon, Docker, o instalación nativa — lo que ya uses). Crear la base `bajonea` vacía. phpMyAdmin se usa como **herramienta de inspección visual**, igual que ya usás DBeaver para el modelo completo; Flyway es quien efectivamente crea las tablas.

2.2 Diseñar el modelo relacional recortado del MVP

Antes de escribir el primer script de Flyway, hay que producir un **mini-diccionario de datos del MVP** (subconjunto del `diccionario-de-datos.md` completo), para no arrastrar columnas o relaciones que no se usan.

Qué pedirle a Claude Code:

"En base al diccionario de datos completo del proyecto y las entidades que definimos que **Sí** entran al MVP (sección 0), generá un archivo `docs/modelo-mvp.md` con: la lista de tablas del MVP, sus columnas (nombre, tipo, nullable, default), sus PK/FK, y sus relaciones. Incluí `Provincia` y `Localidad` tal cual están documentadas en el diccionario completo (catálogo estático desde Georef). `Direccion` debe llevar el modelo completo: calle, numero, piso_depto (nullable), codigo_postal, localidad_id (FK), cliente_id/comercio_id mutuamente excluyentes, principal, eliminada. Incluí `ImagenProducto` tal cual está en el diccionario completo: id, producto_id (FK), url, orden (default 0), es_principal (default false), máximo 5 imágenes por producto validado a nivel aplicación. No incluyas `Pago`, `CuentaMercadoPago`, `NotaCredito`, `Sesion`, `Soporte`, `Reclamo`, `ConfiguracionTarifa`, ni las tablas de Historial."

Qué validar vos: revisar `docs/modelo-mvp.md` a mano antes de avanzar. Es el documento que después se traduce 1:1 a Flyway y a Entities — un error acá se propaga a todo el resto.

2.3 Migraciones Flyway

Convención de nombres: `V1__init_usuarios.sql`, `V2__init_comercios.sql`, etc. — una migración por bloque funcional coherente, no una sola migración gigante ni una por tabla suelta.

Orden sugerido de migraciones (respeta las dependencias de FK):

1. `V1__geografia.sql` — `provincia`, `localidad`. **Debe ir primero:** `Direccion` depende de `Localidad` vía FK, así que este catálogo tiene que existir antes de cualquier tabla de identidad.
2. `V2__usuarios_y_personas.sql` — `usuario`, `persona_fisica`, `persona_juridica`.
3. `V3__roles.sql` — `cliente`, `comercio`, `administrador`.
4. `V4__direcciones.sql` — `direccion` (FK a `localidad`, a `cliente` y a `comercio`).
5. `V5__tokens.sql` — `token`.
6. `V6__categorias_y_tags.sql` — `categoria`, `tag`.
7. `V7__productos.sql` — `producto`, `imagen_producto`, `producto_tag`.
8. `V8__carrito.sql` — `carrito`, `item_carrito`.
9. `V9__pedidos.sql` — `pedido`, `detalle_pedido`.
10. `V10__notificaciones.sql` — `notificacion`.

Qué pedirle a Claude Code: generar cada script a partir de `docs/modelo-mvp.md`, uno por uno, mostrándote el SQL antes de aplicarlo (esto es plan-mode natural: proponer y esperar validación en pasos con impacto estructural, según el PDF sección 2.4).

2.4 Aplicar migraciones y verificar en phpMyAdmin

Qué pedirle a Claude Code: levantar la app (`./mvnw spring-boot:run`) para que Flyway aplique las migraciones contra `bajonea`. En este punto `provincia` y `localidad` existen como tablas pero están **vacías** — el dato se carga recién en la Fase 2bis, no en la migración de estructura.

Qué validar vos: entrar a phpMyAdmin, confirmar que las 10+ tablas existen con las columnas esperadas, y que `flyway_schema_history` tiene una fila por migración aplicada.

Checklist de cierre de Fase 2

- ☐ `docs/modelo-mvp.md` aprobado.
 - ☐ Migraciones V1 a V10 (o las que correspondan) aplicadas sin error.
 - ☐ Tablas visibles y correctas en phpMyAdmin.
 - ☐ Ningún dato ni tabla del alcance completo (Pago, Reclamo, etc.) presente.
-

Fase 2bis — Carga del catálogo geográfico (ETL Georef)

Esta fase es nueva respecto de la primera versión de la guía: como el registro de Cliente y de Comercio exige seleccionar Provincia → Localidad, **no se puede probar ni un registro** sin este catálogo cargado. Va inmediatamente después de la Fase 2 y antes de tocar cualquier Entity.

2bis.1 Naturaleza del proceso

Según `requisitos-funcionales-generales.md`: la carga se hace **una sola vez, como tarea de configuración inicial**, no como una operación disponible en runtime para ningún rol. No es un endpoint de la API — es un script de infraestructura que se corre una vez (y ocasionalmente se vuelve a correr a mano si Georef publica localidades nuevas, algo fuera del flujo operativo).

2bis.2 Alcance de la carga para el MVP

El diccionario completo prevé cobertura de **todo el país**. Para el MVP alcanza con no recortar esto — es la misma carga completa que se necesitaría en producción, así que no tiene sentido cargar "solo Tierra del Fuego" y tener que rehacer el ETL después. Se carga el país completo desde el arranque.

2bis.3 Qué pedirle a Claude Code

"Escribí un script (Java, ejecutable como `CommandLineRunner` detrás de un flag `--cargar-geografia`, o un script standalone en `scripts/etl-georef/`, lo que prefieras vos) que:

1. Llame a la API Georef (`https://apis.datos.gob.ar/georef/api/provincias` y `.../localidades`, paginando con `campos=estandar` y `max/inicio`).
2. Inserte cada provincia en la tabla `provincia` (id = código Georef, nombre).
3. Inserte cada localidad en `localidad` (id = código Georef, nombre, `provincia_id`).
4. Sea idempotente (si se corre dos veces, no duplica ni falla — usar `INSERT ... ON DUPLICATE KEY UPDATE` o verificar existencia antes de insertar)."

Nota de red: confirmar primero que el dominio de la API Georef esté alcanzable desde donde vayas a correr el script (en el entorno de Claude Code puede estar restringido por la configuración de red; si es así, correrlo desde tu propia máquina/IDE en lugar de pedírselo a Claude Code directamente).

2bis.4 Validar la carga

Qué validar vos en phpMyAdmin: `SELECT COUNT(*) FROM provincia` debería dar 24 (23 provincias + CABA); `SELECT COUNT(*) FROM localidad` debería dar varios miles. Verificar puntualmente que Tierra del Fuego y Río Grande estén presentes, ya que son los datos que vas a usar en tus propias pruebas manuales y en los seeds de testing.

Checklist de cierre de Fase 2bis

- [] `provincia` cargada completa (24 registros).
- [] `localidad` cargada completa (miles de registros, sin duplicados).
- [] Río Grande / Tierra del Fuego verificados presentes y correctos.
- [] El script es re-ejecutable sin romper nada (idempotente).

Fase 3 — Estructura de capas y convenciones de código

3.1 Estructura de paquetes

Confirmando y afinando tu propuesta — es correcta, con una precisión: **sí, la lógica de negocio va en `services`; `repositories` debe quedar "tonto" (solo queries derivadas o `@Query`, sin reglas de negocio adentro), y `controllers` debe quedar aún más tonto (solo mapear HTTP ↔ DTO y delegar al service).** Esa separación es la que evita el problema típico de "god controller" o "god repository".

```
com.bajonea.backend/
├─ BajoneaApplication.java
├─ config/
│   └─ security/
│       ├── SecurityConfig.java          <- cadena de filtros, reglas por endpoint
│       ├── JwtAuthenticationFilter.java
│       └─ JwtService.java              <- generar/validar/parsear el token
│   └─ authentication/
│       └─ (si hace falta separar detalles de autenticación de autorización)
├─ OpenApiConfig.java
└─ MailConfig.java
```

- └─ CloudinaryConfig.java <- bean del cliente Cloudinary con credenciales
- └─ dto/
 - └─ request/
 - └─ RegistroClienteRequestDTO.java
 - └─ RegistroComercioRequestDTO.java
 - └─ DireccionRequestDTO.java <- calle, numero, pisoDepto, codigoPostal, localidad
 - └─ LoginRequestDTO.java
 - └─ ProductoRequestDTO.java
 - └─ ImagenProductoRequestDTO.java <- url (post-Cloudinary), orden, esPrincipal
 - └─ CategoriaRequestDTO.java
 - └─ TagRequestDTO.java
 - └─ ItemCarritoRequestDTO.java
 - └─ PedidoRequestDTO.java
 - └─ RechazoPedidoRequestDTO.java
 - └─ response/
 - └─ ApiResponse.java <- envoltorio estándar {mensaje, data}
 - └─ UsuarioResponseDTO.java
 - └─ ComercioResponseDTO.java
 - └─ ProductoResponseDTO.java
 - └─ ImagenProductoResponseDTO.java
 - └─ CategoriaResponseDTO.java
 - └─ TagResponseDTO.java
 - └─ CarritoResponseDTO.java
 - └─ PedidoResponseDTO.java
 - └─ NotificacionResponseDTO.java
 - └─ ProvinciaResponseDTO.java
 - └─ LocalidadResponseDTO.java
 - └─ CloudinarySignatureResponseDTO.java <- firma de subida para el frontend
- └─ entities/
 - └─ Provincia.java
 - └─ Localidad.java
 - └─ Usuario.java
 - └─ PersonaFisica.java
 - └─ PersonaJuridica.java
 - └─ Cliente.java
 - └─ Comercio.java
 - └─ Administrador.java
 - └─ Direccion.java
 - └─ Token.java
 - └─ Categoria.java
 - └─ Tag.java
 - └─ Producto.java
 - └─ ImagenProducto.java
 - └─ ProductoTag.java
 - └─ Carrito.java
 - └─ ItemCarrito.java
 - └─ Pedido.java
 - └─ DetallePedido.java
 - └─ Notificacion.java
 - └─ enums/
 - └─ RolUsuario.java
 - └─ EstadoUsuario.java
 - └─ EstadoComercio.java

```

|         └─ EstadoProducto.java
|         └─ EstadoPedido.java
|         └─ TipoEntrega.java
|         └─ TipoToken.java
|         └─ MotivoRechazo.java
└─ repositories/
    └─ ProvinciaRepository.java
    └─ LocalidadRepository.java
    └─ UsuarioRepository.java
    └─ ClienteRepository.java
    └─ ComercioRepository.java
    └─ AdministradorRepository.java
    └─ DireccionRepository.java
    └─ TokenRepository.java
    └─ CategoriaRepository.java
    └─ TagRepository.java
    └─ ProductoRepository.java
    └─ ImagenProductoRepository.java
    └─ CarritoRepository.java
    └─ ItemCarritoRepository.java
    └─ PedidoRepository.java
    └─ DetallePedidoRepository.java
    └─ NotificacionRepository.java
└─ services/
    └─ AuthService.java
    └─ RegistroService.java
    └─ GeografiaService.java          <- listar provincias/localidades para los selectores
    └─ ComercioService.java
    └─ AdministradorService.java
    └─ CategoriaService.java
    └─ TagService.java
    └─ ProductoService.java
    └─ CarritoService.java
    └─ PedidoService.java
    └─ NotificacionService.java
    └─ EmailService.java
    └─ CloudinaryService.java        <- generar firma de subida, validar límite de imagen
└─ controllers/
    └─ AuthController.java
    └─ ComercioController.java
    └─ AdministradorController.java
    └─ CategoriaController.java
    └─ TagController.java
    └─ ProductoController.java
    └─ CatalogoController.java       <- endpoints públicos de exploración
    └─ GeografiaController.java      <- GET /provincias, GET /localidades?provinciaId=
    └─ CarritoController.java
    └─ PedidoController.java
    └─ NotificacionController.java
└─ exceptions/
    └─ GlobalExceptionHandler.java  <- @RestControllerAdvice
    └─ RecursoNoEncontradoException.java
    └─ ConflictoDeNegocioException.java

```

```
| └─ ValidacionException.java
└─ mappers/                                     <- opcional: si se prefiere no mapear a mano en el s
    └─ (uno por entidad si se decide usar MapStruct)
```

Nota sobre `mappers/` : vos preguntaste dónde va la conversión Entity↔DTO. Dos caminos válidos: (a) mapeo manual dentro del propio `Service` (más simple, cero dependencias nuevas, recomendado para un TFC donde también se evalúa que entendés cada capa), o (b) MapStruct con una carpeta `mappers/` . Para este proyecto, dado que ya tenés Lombok y el volumen de entidades es manejable, se recomienda **mapeo manual en el Service** — es más explícito para defender en la evaluación académica y evita una dependencia más que aprender bajo presión de tiempo.

3.2 Convención de nombres

- Entities: sustantivo singular (`Producto` , no `Productos`).
- DTOs: `<Entidad><Acción>DTO` para requests (`ProductoRequestDTO`), `<Entidad>ResponseDTO` para responses.
- Services: interfaz + implementación solo si hay más de una implementación real prevista; si no, clase concreta directa (no sobre-diseñar con interfaces que van a tener un único implementador).
- Controllers: `@RestController` + `@RequestMapping("/api/v1/<recurso-en-plural>")` .

3.3 Convención de respuesta HTTP estándar

```
public class ApiResponse<T> {
    private String mensaje;
    private T data;
    // constructores, getters
}
```

Todo controller devuelve `ResponseEntity<ApiResponse<XxxResponseDTO>>` (o `ApiResponse<List<XxxResponseDTO>>` para listados). Esto resuelve directamente tu duda: nunca se expone la entidad, siempre DTO envuelto, y siempre hay mensaje.

Checklist de cierre de Fase 3

- ☐ Paquetes creados (vacíos, con al menos un `package-info.java` o clase placeholder si hace falta).
- ☐ `ApiResponse<T>` implementado y probado con un endpoint dummy (`GET /api/v1/health`).
- ☐ `GlobalExceptionHandler` mapea excepciones propias a los códigos HTTP de la sección 0.4.

Fase 4 — Entities (JPA)

4.1 Estrategia de herencia Usuario/Persona

El modelo completo usa **tabla compartida por PK** (`Persona` 1:1 `Usuario` , con `PersonaFisica` / `PersonaJuridica` como subtipos). Para el MVP se mantiene esta estrategia porque es central al dominio (Cliente y Comercio comparten la noción de "Persona" pero con atributos distintos) — simplificarla de más generaría más trabajo de refactor a futuro que el que ahorra ahora.

Mapeo JPA recomendado: `@Entity + @Inheritance(strategy = InheritanceType.JOINED)` sobre `Persona` , o bien composición explícita con `@OneToOne + @MapsId` (más cercano a como está documentado el modelo con "PK idéntica"). **Recomendado:** `@MapsId` , porque es más explícito y más fácil de razonar en un modelo con exclusión mutua (una Persona es Física O Jurídica, nunca ambas) que la herencia JPA pura.

4.2 Lista de entidades a crear (con atributos clave del MVP)

Generar una por una, no todas en un solo prompt gigante — permite revisar cada una contra `docs/modelo-mvp.md` antes de seguir. `Provincia` / `Localidad` van primero porque `Direccion` depende de `Localidad` .

1. **Provincia:** id (String, código Georef, PK), nombre. Sin `@GeneratedValue` — el ID lo provee el ETL de la Fase 2bis, no la aplicación.
2. **Localidad:** id (String, código Georef, PK), nombre, provincia (`@ManyToOne a Provincia`). Igual que `Provincia` , sin autogenerado.
3. **Usuario:** id, email (unique), passwordHash, rol (enum `RolUsuario`), estado (enum `EstadoUsuario` , default `PENDIENTE`), fechaCreacion.
4. **PersonaFisica:** id (=usuario.id), nombre, apellido, dni (unique), fechaNacimiento, telefono.
5. **PersonaJuridica:** id (=usuario.id), razonSocial, cuit (unique), tipoPersonaJuridica (enum, ver nota abajo).
6. **Cliente:** id (=personaFisica.id), relación a `Direccion` (1:N, aunque el MVP solo usa la primera).
7. **Comercio:** id, personaJuridica (N:1 o 1:1 según regla de negocio "1 comercio por PersonaJuridica" a nivel aplicación), nombreComercio, descripcion, tipoComercio (enum), estado (enum `EstadoComercio` , default `PENDIENTE`), direccion (1:1).
8. **Administrador:** id (=personaFisica.id). Sin atributos adicionales en el MVP.
9. **Direccion:** id, calle, numero, pisoDepto (nullable), codigoPostal, localidad (`@ManyToOne a Localidad` , obligatorio), clienteId (nullable), comercioId (nullable, unique), principal (boolean, solo aplica si `clienteId` no es null), eliminada (boolean, baja lógica). Una sola tabla con ambas FK nullable, mismo patrón que el modelo completo, documentando la regla de exclusión mutua a nivel aplicación (no a nivel constraint SQL, salvo que se quiera reforzar con un `CHECK` — MySQL 8 sí los soporta).
10. **Token:** id, usuarioId, token (UUID, unique), tipo (enum, en MVP solo `VERIFICACION_EMAIL`), fechaExpiracion, usado (boolean).
11. **Categoria:** id, nombre (unique), activo (boolean).
12. **Tag:** id, nombre (unique), activo (boolean).
13. **Producto:** id, comercioId, categoriaId, nombre, descripcion, precio (BigDecimal), estado (enum `EstadoProducto` : DISPONIBLE/AGOTADO/DESCONTINUADO), fechaCreacion,

fechaModificacion. **Ya no lleva columna de imagen** — la galería vive en `ImagenProducto` (relación 1:N), ver punto siguiente.

14. **ImagenProducto**: id, productold (FK), url (String, URL devuelta por Cloudinary tras la subida firmada — ver Fase 11), orden (int, default 0), esPrincipal (boolean, default false, solo una imagen por producto puede tener `true`). Máximo 5 filas por `productoId`, validado a nivel aplicación (`ProductoService / ImagenProductoService`), no a nivel constraint SQL.
15. **ProductoTag**: PK compuesta (productold, tagId).
16. **Carrito**: id, clientold (unique — un carrito por cliente), comerciold (nullable — vacío hasta el primer ítem).
17. **ItemCarrito**: id, carritold, productold, cantidad, nota (opcional).
18. **Pedido**: id, clientold, comerciold, tipoEntrega (enum `DOMICILIO / RETIRO`), direccionId (nullable, solo si `DOMICILIO`), estado (enum `EstadoPedido` **recortado**: `PENDIENTE`, `ACEPTADO`, `RECHAZADO`), motivoRechazo (enum, nullable), comentarioRechazo (texto, nullable), subtotal, fechaCreacion.
19. **DetallePedido**: id, pedidold, productold, precioUnitario (snapshot, immutable), cantidad, subtotal.
20. **Notificacion**: id, usuariold, mensaje, leida (boolean, default false), fechaCreacion.

Nota sobre `Provincia / Localidad` como entidades de solo lectura: en la capa de repositorio no exponer métodos de escritura más allá de los que use el propio ETL (que puede, si se prefiere, insertar directo por JDBC/SQL en vez de pasar por el `Repository` de JPA — es una decisión razonable ya que es un proceso de infraestructura, no de aplicación). Los `Service / Controller` de cara a la aplicación solo necesitan lectura (`findAll`, `findByProvinciaId`).

Nota sobre `TipoPersonaJuridica`: el catálogo completo tiene 18 valores (SA, SRL, SAS, etc.). Para el MVP, evaluar con vos si conviene el enum completo o un subconjunto reducido (ej. solo `MONOTRIBUTO / RESPONSABLE_INSCRIPTO / SAS / SRL / OTRO`) ya que no afecta lógica de negocio en esta etapa, solo almacenamiento. Recomendación: mantener el enum completo igual, porque no cuesta nada extra (es un campo más) y evita tener que migrar el dato después.

Nota sobre `EstadoPedido` **recortado:** el modelo completo tiene 8 estados (`PENDIENTE_PAGO`, `PENDIENTE`, `EN_PREPARACION`, `EN_CAMINO`, `LISTO_PARA_RETIRAR`, `ENTREGADO`, `RECHAZADO`, `ANULADO`, `EXPIRADO`, `CANCELADO_POR_SISTEMA`). El MVP definido en la imagen adjunta llega hasta "aceptar o rechazar el pedido y notificar" — no incluye el flujo de preparación/entrega. Por eso el enum del MVP queda en 3 valores: `PENDIENTE` → `ACEPTADO` / `RECHAZADO`. Si en una iteración posterior se quiere sumar "en preparación → entregado" es una migración Flyway adicional más un ajuste de enum, no un rediseño.

4.3 Reglas de mapeo comunes a todas las entities

- `@Entity`, `@Table(name = "...")` explícito (nombres en `snake_case` en BD, `camelCase` en Java).
- `@Id @GeneratedValue(strategy = GenerationType.IDENTITY)` salvo en las que usan `@MapsId`.
- Lombok: `@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder` — evaluar no usar `@Data` a secas porque genera `equals / hashCode` sobre todos los campos, lo cual es problemático en entidades JPA con relaciones bidireccionales (riesgo de recursión infinita). Usar `@EqualsAndHashCode(of = "id")` explícito.

- Relaciones `@ManyToOne` con `fetch = FetchType.LAZY` siempre, salvo justificación puntual.
- Nunca exponer estas entidades directamente como tipo de retorno en un `@RestController` (hook lo va a validar, ver Anexo C).

Checklist de cierre de Fase 4

- [] Las 20 entidades creadas y compilando (incluye `Provincia`, `Localidad` e `ImagenProducto`).
 - [] Cada entidad corresponde 1:1 a una tabla ya migrada en Flyway (Fase 2) y con datos si corresponde (Fase 2bis para geografía).
 - [] `mvn spring-boot:run` levanta sin error de mapeo Hibernate (`ddl-auto=validate` es quien lo detecta).
 - [] `Direccion` mapea correctamente `localidad` como `@ManyToOne`, no como columna suelta.
 - [] `Producto` no tiene ninguna columna de imagen propia — la relación a `ImagenProducto` es `@OneToMany`.
-

Fase 5 — DTOs y mapeo

5.1 Principio general

Un DTO de request nunca incluye campos que el cliente no debe poder setear (`id`, `estado`, `fechaCreacion`). Un DTO de response nunca incluye campos sensibles (`passwordHash`) ni expone relaciones completas innecesarias (ej. `ProductoResponseDTO` no necesita el `ComercioResponseDTO` completo adentro, alcanza con `comercioId` y `nombreComercio`).

5.2 Lista de DTOs a crear

Requests:

- `DireccionRequestDTO` (calle, numero, pisoDepto opcional, codigoPostal, localidadId). Se anida dentro de los DTOs de registro y se reutiliza tal cual si más adelante se agrega gestión de direcciones desde el perfil.
- `RegistroClienteRequestDTO` (nombre, apellido, dni, fechaNacimiento, telefono, email, password, `direccion: DireccionRequestDTO`).
- `RegistroComercioRequestDTO` (razonSocial, cuit, tipoPersonaJuridica, nombreComercio, descripcion, tipoComercio, email, password, telefono, `direccion: DireccionRequestDTO`).
- `LoginRequestDTO` (email, password).
- `CategoriaRequestDTO` (nombre).
- `TagRequestDTO` (nombre).
- `ProductoRequestDTO` (nombre, descripcion, precio, categoriaId, tagIds: `List<Long>`, `imagenes: List<ImagenProductoRequestDTO>` — cada una con la URL ya devuelta por Cloudinary, orden y si es la principal; el backend no recibe el archivo binario en ningún momento).
- `ImagenProductoRequestDTO` (url, orden, esPrincipal) — se usa tanto anidado en `ProductoRequestDTO` al crear/editar como de forma independiente si se agrega una imagen suelta

a un producto ya existente.

- `ItemCarritoRequestDTO` (`productId`, `cantidad`, `nota`).
- `PedidoRequestDTO` (`tipoEntrega`, `direccionId` opcional).
- `RechazoPedidoRequestDTO` (`motivoRechazo`, `comentario` opcional/obligatorio si `motivo=OTRO`).
- `AprobacionComercioRequestDTO` (`aprobar`: `boolean`, `motivoRechazo` opcional).

Responses:

- `ApiResponse<T>` (genérico, ya definido en 3.3).
- `UsuarioResponseDTO` (`id`, `email`, `rol`, `estado` — sin `password`).
- `DireccionResponseDTO` (`id`, `calle`, `numero`, `pisoDepto`, `codigoPostal`, `localidad`: `nombre`, `provincia`: `nombre`, `principal`).
- `ComercioResponseDTO` (`id`, `nombreComercio`, `descripcion`, `tipoComercio`, `estado`, `direccion`: `DireccionResponseDTO`).
- `ProductoResponseDTO` (`id`, `nombre`, `descripcion`, `precio`, `categoria`, `tags`, `imagenes`: `List<ImagenProductoResponseDTO>` , `estado`).
- `ImagenProductoResponseDTO` (`id`, `url`, `orden`, `esPrincipal`).
- `CategoriaResponseDTO` , `TagResponseDTO` (`id`, `nombre`, `activo`).
- `CarritoResponseDTO` (`comercioId`, `nombreComercio`, `items`: `List<ItemCarritoResponseDTO>` , `subtotal`).
- `PedidoResponseDTO` (`id`, `estado`, `tipoEntrega`, `detalles`: `List<DetallePedidoResponseDTO>` , `subtotal`, `fecha`).
- `NotificacionResponseDTO` (`id`, `mensaje`, `leida`, `fecha`).
- `ProvinciaResponseDTO` (`id`, `nombre`).
- `LocalidadResponseDTO` (`id`, `nombre`, `provincialId`).
- `CloudinarySignatureResponseDTO` (`signature`, `timestamp`, `apiKey`, `cloudName`, `folder`) — todo lo que el frontend necesita para subir directo a Cloudinary sin exponer el `api_secret` .

5.3 Validación de entrada

Usar `jakarta.validation` (`@NotBlank` , `@Email` , `@Positive` , `@Size`) directamente en los DTOs de request, y `@Valid` en la firma del método del controller. El `GlobalExceptionHandler` captura `MethodArgumentNotValidException` y devuelve `400` con el `ApiResponse` estándar listando los campos inválidos en el mensaje.

Checklist de cierre de Fase 5

- [] Todos los DTOs de la lista 5.2 creados.
- [] Ningún DTO de response incluye `passwordHash` ni la entidad completa anidada donde no corresponde.
- [] Validaciones Bean Validation puestas en cada request DTO.

Fase 6 — Repositories

6.1 Principio

Extender `JpaRepository<Entidad, Long>` (o el tipo de PK que corresponda). Agregar métodos derivados o `@Query` solo cuando el service realmente los necesite — no generar de antemano getters que nadie usa.

6.2 Métodos concretos que se van a necesitar (guía, no exhaustiva)

- `UsuarioRepository`: `findByEmail(String email)`, `existsByEmail(String email)`.
- `PersonaFisicaRepository` (si se separa de `Cliente/Administrador`): `existsByDni(String dni)`.
- `PersonaJuridicaRepository`: `existsByCuit(String cuit)`.
- `ComercioRepository`: `findByEstado(EstadoComercio estado)` — para el catálogo público y para el listado de "pendientes de aprobación" del admin.
- `TokenRepository`: `findByTokenAndUsado(String token, boolean usado)`.
- `ProductoRepository`: `findByComercioIdAndEstadoNot(Long comercioId, EstadoProducto estado)`, `findByComercioIdAndCategoriaId(...)` para el filtro de menú.
- `ImagenProductoRepository`: `findByProductoIdOrderByOrdenAsc(Long productoId)`, `countByProductoId(Long productoId)` (para validar el máximo de 5 antes de insertar una nueva).
- `CarritoRepository`: `findByClienteId(Long clienteId)`.
- `PedidoRepository`: `findByComercioIdAndEstado(...)`, `findByClienteId(...)`.
- `NotificacionRepository`: `findByUsuarioIdOrderByFechaCreacionDesc(Long usuarioId)`, `findByUsuarioIdAndLeidaFalse(Long usuarioId)`.

Checklist de cierre de Fase 6

- [] Todos los repositories creados con al menos las queries listadas.
 - [] Cada repository devuelve tipos de colección `List<T>` (nunca `Set` salvo justificación puntual, según la regla transversal).
-

Fase 7 — Configuración: seguridad, JWT y autorización por rol

Esta es la carpeta `config/` que mencionaste: `authentication` y `authorization` son dos caras de la misma cadena de filtros de Spring Security, no necesariamente dos paquetes separados — pero si preferís separarlos físicamente para mayor claridad académica, es válido

(`config/security/authentication/JwtService.java` + `config/security/authorization/SecurityConfig.java`).

7.1 `JwtService`

Responsable de: generar el token al login (claims: `sub=email`, `rol`, `userId`, expiración), validarlo, y extraer claims. Usa `jwt.secret` y `jwt.expiration-ms` de `application.properties`.

7.2 `JwtAuthenticationFilter`

`OncePerRequestFilter` que lee el header `Authorization: Bearer <token>`, valida, y si es válido carga el `Authentication` en el `SecurityContextHolder` con el rol como `GrantedAuthority` (ej. `ROLE_CLIENTE`, `ROLE_COMERCIO`, `ROLE_ADMINISTRADOR`).

7.3 SecurityConfig

Define:

- Endpoints públicos (sin auth): `POST /api/v1/auth/registro/cliente`, `POST /api/v1/auth/registro/comercio`, `POST /api/v1/auth/login`, `GET /api/v1/auth/verificar/{token}`, `GET /api/v1/catalogo/**` (exploración pública de comercios y menús), `GET /api/v1/geografia/provincias`, `GET /api/v1/geografia/localidades` (necesarios en las pantallas de registro, que son públicas), Swagger UI.
- Endpoints por rol: `/api/v1/productos/**` (COMERCIO), `/api/v1/categorias/**` y `/api/v1/tags/**` y `/api/v1/administrador/**` (ADMINISTRADOR), `/api/v1/carrito/**` y `/api/v1/pedidos/cliente/**` (CLIENTE), `/api/v1/pedidos/comercio/**` (COMERCIO).
- `sessionCreationPolicy(SessionCreationPolicy.STATELESS)` — coherente con JWT sin sesión server-side.
- CORS habilitado para el origen del frontend (Fase 15) durante desarrollo.

7.4 BCryptPasswordEncoder

Bean único, usado en `RegistroService` para hashear y en `AuthService` para comparar en el login.

Checklist de cierre de Fase 7

- [] Login exitoso devuelve un JWT válido con rol embebido.
 - [] Un endpoint protegido de CLIENTE devuelve `403` si se llama con token de COMERCIO.
 - [] Un endpoint protegido devuelve `401` sin token o con token vencido/corrupto.
 - [] Endpoints públicos (catálogo, registro, login, verificación) accesibles sin token.
-

Fase 8 — Services (lógica de negocio)

Confirmando tu entendimiento: sí, es correcto — acá vive toda la lógica de negocio, las validaciones de reglas (no las de formato, esas ya las resuelve Bean Validation en el DTO), las transiciones de estado, el armado de notificaciones, y el mapeo Entity↔DTO manual (según lo definido en 3.1).

8.1 RegistroService

- `registrarCliente(RegistroClienteRequestDTO)` : valida email/DNI únicos, valida que `localidadId` exista (si no, `404 / 400` claro — nunca dejar que una FK inválida llegue a Hibernate como excepción cruda), crea `Usuario` (estado `PENDIENTE`), `PersonaFisica`, `Cliente`, `Direccion`

inicial marcada `principal = true`, genera `Token` tipo `VERIFICACION_EMAIL`, dispara

`EmailService.enviarVerificacion(...)`. Devuelve `UsuarioResponseDTO`.

- `registrarComercio(RegistroComercioRequestDTO)`: análogo, crea `Usuario` (`PENDIENTE`), `PersonaJuridica`, `Comercio` (estado `EstadoComercio.PENDIENTE`), `Direccion` (única, principal no aplica), token de verificación, email.

8.1bis GeografiaService

- `listarProvincias()`: `ProvinciaRepository.findAll()` mapeado a `List<ProvinciaResponseDTO>` — sin lógica adicional, es un catálogo de solo lectura.
- `listarLocalidadesPorProvincia(String provinciaId)`: `LocalidadRepository.findByProvinciaId(provinciaId)` — usado por el selector dependiente del formulario de registro (Fase 16).

8.2 AuthService

- `login(LoginRequestDTO)`: busca por email, valida `Usuario.estado == ACTIVO` (si está `PENDIENTE`, devolver 403 con mensaje claro de "verificá tu cuenta"), valida password con `BCrypt`, genera JWT vía `JwtService`.
- `verificarCuenta(String token)`: busca `Token` válido y no usado, valida expiración, marca `Usuario.estado = ACTIVO`, marca `Token.usado = true`.

8.3 ComercioService / AdministradorService

- `AdministradorService.listarComerciosPendientes()`: `ComercioRepository.findByEstado(PENDIENTE)`.
- `AdministradorService.resolverAprobacion(Long comercioId, AprobacionComercioRequestDTO)`: si aprueba → estado = `APROBADO`; si rechaza → estado = `RECHAZADO` + guarda motivo. Dispara notificación al comercio (in-app).
- `AdministradorService.gestionarCategoria(...)` / `gestionarTag(...)`: CRUD simple con activo/inactivo (baja lógica, no física — coherente con el resto del proyecto).

8.4 ProductoService

CRUD completo para el comercio autenticado (validar que el `comercioId` del producto coincide con el comercio del JWT antes de permitir editar/dar de baja — regla de autorización a nivel de dato, no solo de rol). Transiciones de estado `DISPONIBLE` ↔ `AGOTADO`, y → `DESCONTINUADO` (irreversible). Al pasar a `AGOTADO` o `DESCONTINUADO`: eliminar el ítem de los carritos activos que lo contengan (mismo patrón documentado en el proyecto completo) y notificar al cliente afectado — esto sí entra al MVP porque es barato de implementar y evita bugs visibles (carrito con producto fantasma).

Gestión de la galería (`ImagenProducto`): al crear o editar un producto, el service recibe la

`List<ImagenProductoRequestDTO>` y:

- Valida que no haya más de 5 imágenes en total para ese producto (contando las ya existentes en edición + las nuevas).
- Valida que **como mucho una** tenga `esPrincipal = true`; si ninguna la tiene, la primera por `orden` se toma como principal por defecto.
- Persiste cada una como una fila de `ImagenProducto` con su `orden`.
- Permite operaciones individuales además del guardado conjunto: agregar una imagen suelta (respetando el máximo de 5), eliminar una imagen puntual, y cambiar cuál es la principal — útil para la pantalla de edición de producto donde el comercio agrega/saca fotos sin recrear todo el producto.

8.5 CarritoService

- `agregarItem(clienteId, ItemCarritoRequestDTO)` : si el carrito está vacío, setea `comercioId` del producto; si ya tiene `comercioId` distinto, lanza `ConflictoDeNegocioException` → 409. Válida cantidad 1-20.
- `verCarrito(clienteId)` , `actualizarCantidad(...)` , `eliminarItem(...)` , `vaciarCarrito(clienteId)` .

8.6 PedidoService

- `confirmarPedido(clienteId, PedidoRequestDTO)` : toma el carrito activo, valida que no esté vacío, crea `Pedido` (`estado = PENDIENTE`) + `DetallePedido` (snapshot de `precioUnitario` desde `Producto.precio` , inmutable desde este punto), calcula `subtotal` , vacía el carrito, notifica al comercio ("Nuevo pedido recibido").
- `resolverPedido(comercioId, pedidoId, aceptar: boolean, RechazoPedidoRequestDTO opcional)` : valida que el pedido pertenece al comercio autenticado y está en `PENDIENTE` . Si acepta → `estado = ACEPTADO` , notifica al cliente. Si rechaza → `estado = RECHAZADO` + motivo, notifica al cliente.
- `listarPedidosCliente(clienteId)` , `listarPedidosComercio(comercioId)` .

8.7 NotificacionService

- `crear(usuarioId, mensaje)` : helper interno usado por los demás services.
- `listar(usuarioId)` , `marcarLeida(notificacionId)` , `contarNoLeidas(usuarioId)` (para el badge de polling).

8.8 EmailService

Encapsula `JavaMailSender` . Un único método relevante al MVP: `enviarVerificacion(String emailDestino, String token)` . Ver detalle SMTP en Fase 10.

Checklist de cierre de Fase 8

- [] Cada service tiene su propia responsabilidad, sin lógica de negocio filtrada a controllers o repositories.
- [] Las reglas de negocio de "regla de negocio del proyecto completo" que SÍ aplican al MVP (email/DNI/CUIT únicos, un producto de un solo comercio en el carrito, snapshot de precio en detalle de pedido) están efectivamente validadas acá, no solo en la base de datos.

Fase 9 — Controllers y contrato de respuestas HTTP

9.1 Principio

Un controller: recibe DTO validado, llama **un** método de service, envuelve el resultado en `ApiResponse`, devuelve `ResponseEntity` con el status correcto. Nada de lógica condicional de negocio acá.

9.2 Ejemplo de referencia (patrón a replicar en todos)

```
@PostMapping("/registro/cliente")
public ResponseEntity<ApiResponse<UsuarioResponseDTO>> registrarCliente(
    @Valid @RequestBody RegistroClienteRequestDTO request) {
    UsuarioResponseDTO usuario = registroService.registrarCliente(request);
    ApiResponse<UsuarioResponseDTO> body =
        new ApiResponse<>("Cuenta creada. Revisá tu email para verificarla.", usuario);
    return ResponseEntity.status(HttpStatus.CREATED).body(body);
}
```

Esto responde directamente tu duda: se usa `HttpStatus.CREATED` (equivalente al `HttpStatus.CREATED / status 201`) para creación, y siempre se devuelve `ApiResponse<DTO>`, nunca la entidad.

9.3 Endpoints por controller (lista de referencia para el MVP)

- `AuthController`: `POST /registro/cliente`, `POST /registro/comercio`, `POST /login`, `GET /verificar/{token}`, `POST /logout` (invalidación del lado cliente — con JWT stateless, el logout real es borrar el token en el frontend; el endpoint puede ser simbólico o, si se prefiere algo más real, llevar una lista corta de tokens invalidados en memoria/tabla — **a decidir en el momento, no bloquea el resto**).
- `CatalogoController` (público): `GET /catalogo/comercios` (solo APROBADO), `GET /catalogo/comercios/{id}/productos` (con filtro opcional por categoría/tag).
- `GeografiaController` (público): `GET /geografia/provincias`, `GET /geografia/localidades?provinciaId=X`.
- `ComercioController`: `GET /comercios/perfil`, `PUT /comercios/perfil`.
- `AdministradorController`: `GET /administrador/comercios/pendientes`, `PUT /administrador/comercios/{id}/resolucion`.
- `CategoriaController`: CRUD completo (ADMIN).
- `TagController`: CRUD completo (ADMIN).

- `ProductoController`: **CRUD completo (COMERCIO)**, `PATCH /productos/{id}/estado`, `POST /productos/{id}/imagenes` (agregar una imagen a la galería), `DELETE /productos/{id}/imagenes/{imagenId}`, `PATCH /productos/{id}/imagenes/{imagenId}/principal` (marcar como portada).
- `CarritoController`: `GET /carrito`, `POST /carrito/items`, `PUT /carrito/items/{id}`, `DELETE /carrito/items/{id}`, `DELETE /carrito`.
- `PedidoController`: `POST /pedidos` (**CLIENTE**), `GET /pedidos/cliente` (**CLIENTE**), `GET /pedidos/comercio` (**COMERCIO**), `PATCH /pedidos/{id}/resolucion` (**COMERCIO**, body: aceptar + motivo si rechaza).
- `NotificacionController`: `GET /notificaciones` (usuario autenticado, cualquier rol), `PATCH /notificaciones/{id}/leida`.

Checklist de cierre de Fase 9

- ☐ Cada endpoint de la lista 9.3 implementado.
- ☐ Verificado manualmente con `curl` o Postman que cada uno devuelve el status HTTP correcto según la tabla de la sección 0.4.
- ☐ Ningún endpoint devuelve una entidad JPA en el body (verificable con el hook del Anexo C).

Fase 10 — Email real (SMTP) para verificación de cuenta

Dado que confirmaste que la verificación necesita token real + email real, esto entra al MVP y no puede quedar "a confirmar" como en la documentación del proyecto completo.

10.1 Elegir proveedor SMTP

Recomendación ya documentada en el proyecto: **Brevo** (plan gratuito, 300 emails/día — más que suficiente para el volumen de un TFC en etapa de pruebas). Alternativa: verificar si DonWeb incluye SMTP en el plan de hosting contratado. Esta es una decisión tuya puntual, no bloquea el resto de la guía — se puede avanzar con Brevo en modo sandbox mientras se confirma.

10.2 Configuración

Completar en `application-local.properties` (no versionado) o variables de entorno:

```
SMTP_HOST=smtp-relay.brevo.com
SMTP_PORT=587
SMTP_USER=<tu usuario Brevo>
SMTP_PASSWORD=<tu clave SMTP Brevo>
```

10.3 `EmailService.enviarVerificacion`

Plantilla mínima en texto plano o HTML simple (no hace falta un motor de plantillas como Thymeleaf para un solo email de MVP — se puede armar el HTML como `String` con el link `http://localhost:8080/api/v1/auth/verificar/{token}` o, mejor, un link al **frontend** que a su vez llame al endpoint, para que el usuario no vea un JSON crudo en el navegador).

10.4 Flujo de verificación completo

1. Registro → se crea `Token` (UUID, tipo `VERIFICACION_EMAIL`, expiración ej. 24hs) → se envía email con link.
2. Usuario hace clic → frontend en `/verificar.html?token=xxx` → JS llama `GET /api/v1/auth/verificar/{token}`.
3. Backend valida token no usado y no expirado → `Usuario.estado = ACTIVO`, `Token.usado = true`.
4. Frontend muestra "Cuenta verificada, ya podés iniciar sesión" y redirige a login.

Checklist de cierre de Fase 10

- ☐ Email real llega a una casilla de prueba al registrarse.
 - ☐ El link del email verifica correctamente la cuenta.
 - ☐ Un token ya usado o expirado devuelve `409 / 400` con mensaje claro, no `500`.
 - ☐ Login antes de verificar devuelve `403` con mensaje explícito.
-

Fase 11 — Cloudinary: subida firmada de imágenes de producto

Vos confirmaste que Cloudinary sí entra al MVP con la **galería completa** — es la misma integración y el mismo alcance de imágenes que ya tenías resuelto para el proyecto completo (hasta 5 imágenes por producto, una marcada como principal), no una versión recortada.

11.1 Crear cuenta y obtener credenciales

Plan gratuito de Cloudinary alcanza sobradamente para un TFC. Necesitás `cloud_name`, `api_key`, `api_secret` desde el dashboard.

11.2 Variables de entorno

```
CLOUDINARY_CLOUD_NAME=...
CLOUDINARY_API_KEY=...
CLOUDINARY_API_SECRET=...
```

Nunca commitear el `api_secret` — es lo único que realmente protege que alguien suba archivos arbitrarios a tu cuenta.

11.3 Flujo de subida firmada (el mismo patrón que en el proyecto completo)

1. El frontend, antes de subir cada imagen, pide al backend: `POST /api/v1/productos/{id}/cloudinary/firma` (autenticado como COMERCIO). Se pide **una firma por imagen** a subir, no una firma general para el lote — así el backend puede rechazar la firma número 6 si el producto ya tiene 5.
2. El backend (`CloudinaryService`) valida primero cuántas imágenes tiene ya el producto (`ImagenProductoRepository.countByProductoId`); si ya son 5, devuelve `409` sin llegar a firmar nada.
3. Si hay lugar, genera una firma con `api_secret` + un `timestamp` + parámetros acordados (carpeta, por ejemplo `productos/{comercioId}/{productoId}/`), sin exponer el secreto. Devuelve `CloudinarySignatureResponseDTO` .
4. El frontend sube el archivo **directamente a Cloudinary** (no pasa por el backend) usando esa firma.
5. Cloudinary devuelve la URL pública de la imagen subida.
6. El frontend agrega esa URL a la lista de `ImagenProductoRequestDTO` (con su `orden` y si es la principal) y la manda al backend, que persiste la fila en `ImagenProducto` .

Este flujo se repite hasta 5 veces por producto (una vez por imagen), tanto al crear el producto por primera vez como al agregarle fotos después desde la edición.

11.4 Dónde se valida el límite de 5

El backend valida el límite **antes de emitir cada firma**, no después de la subida (evita que Cloudinary reciba un archivo que después se descarta por superar el máximo). La validación vive en `CloudinaryService.generarFirma(comercioId, productoId)` , apoyada en `ImagenProductoRepository.countByProductoId` . La regla de "solo una imagen principal" se valida en `ProductoService / ImagenProductoService` al momento de persistir, no en Cloudinary (que no sabe nada del concepto de "principal").

11.5 cloudinaryConfig

Bean único del cliente Cloudinary (`com.cloudinary.Cloudinary`), inicializado con las tres variables de entorno de 11.2.

Checklist de cierre de Fase 11

- [] Un producto nuevo puede crearse con hasta 5 imágenes subidas a Cloudinary, todas visibles por URL pública.
 - [] Un sexto intento de subida sobre el mismo producto devuelve `409` antes de llegar a Cloudinary.
 - [] El `api_secret` nunca viaja al frontend ni aparece en ningún response.
 - [] Solo una imagen por producto puede tener `esPrincipal = true` ; marcar una nueva como principal desmarca la anterior.
 - [] Eliminar una imagen puntual de la galería funciona sin afectar las demás ni sus `orden` .
-

Fase 12 — Notificaciones in-app (polling)

12.1 Confirmar el modelo

Ya cubierto en Fase 4/8: tabla `Notificacion` simple, sin canal dual, sin email para pedidos.

12.2 Endpoint de polling

`GET /api/v1/notificaciones` — devuelve las notificaciones del usuario autenticado (extraído del JWT, no de un parámetro de URL manipulable), ordenadas por fecha descendente. El frontend hace polling cada X segundos (Fase 16) contra este endpoint.

12.3 Puntos donde se dispara una notificación en el MVP

- Comercio aprobado/rechazado por admin → notifica al comercio.
- Nuevo pedido recibido → notifica al comercio.
- Pedido aceptado/rechazado → notifica al cliente.
- Producto pasado a AGOTADO/DESCONTINUADO estando en el carrito del cliente → notifica al cliente (si se implementa esta regla de la Fase 8.4).

Checklist de cierre de Fase 12

- ☐ Cada evento de 12.3 genera efectivamente una fila en `Notificacion`.
 - ☐ El polling desde el frontend refleja notificaciones nuevas sin recargar la página.
-

Fase 13 — Documentación automática (springdoc-openapi)

13.1 Configuración mínima

`OpenApiConfig` con título, versión, y definición del esquema de seguridad Bearer JWT para que Swagger UI permita probar endpoints autenticados directamente.

13.2 Uso

Disponible en `http://localhost:8080/swagger-ui.html` una vez implementados los controllers. Sirve como **complemento** de Postman (Fase 14), no como reemplazo: Swagger UI es para explorar/probar manualmente durante el desarrollo; la colección Postman es para validación repetible y versionada del contrato de la API.

Checklist de cierre de Fase 13

- [] Swagger UI levanta y lista todos los endpoints de la Fase 9, incluyendo `GeografiaController` y la firma de Cloudinary.
- [] El botón "Authorize" con Bearer token funciona contra un endpoint protegido real.

Fase 14 — Testing de API con Postman

14.1 Estructura de la colección

Una colección `Bajonea-MVP.postman_collection.json` organizada en carpetas por dominio, en el mismo orden que los controllers:

```
Bajonea MVP/  
├─ 00 - Health  
├─ 01 - Geografía (provincias, localidades — sin auth, se corre primero)  
├─ 02 - Auth (registro cliente, registro comercio, login, verificar)  
├─ 03 - Catálogo (público)  
├─ 04 - Administrador (aprobación comercios, categorías, tags)  
├─ 05 - Productos (comercio, incluye firma Cloudinary)  
├─ 06 - Carrito (cliente)  
├─ 07 - Pedidos (cliente + comercio)  
└─ 08 - Notificaciones
```

14.2 Variables de entorno Postman

Un Environment `Bajonea Local` con: `base_url = http://localhost:8080/api/v1`, `token_cliente`, `token_comercio`, `token_admin` (se completan dinámicamente con scripts de test, ver 14.3), `provincia_id`, `localidad_id` (se completan desde la carpeta 01, se reutilizan en los registros de la carpeta 02), `comercio_id`, `producto_id`, `pedido_id` (para encadenar requests).

14.3 Scripts de test por request (pestaña "Tests" de Postman)

Patrón estándar a aplicar en cada request:

```
pm.test("Status code es el esperado", () => {  
    pm.response.to.have.status(201); // o el que corresponda  
});  
  
pm.test("La respuesta tiene mensaje", () => {  
    const json = pm.response.json();  
    pm.expect(json.mensaje).to.exist;  
});  
  
// Al hacer login, guardar el token para requests siguientes:  
pm.environment.set("token_cliente", pm.response.json().data.token);
```

14.4 Orden de ejecución recomendado (para correr como Collection Runner)

1. GET /geografia/provincias → 200, guarda `provincia_id` (ej. Tierra del Fuego).
2. GET /geografia/localidades?provinciaId=... → 200, guarda `localidad_id` (ej. Río Grande).
3. Registro cliente (con `localidad_id` en la dirección) → 201.
4. Registro comercio (idem) → 201.
5. (Manual, fuera de Postman) verificar ambas cuentas por email real o, si se quiere automatizar, un endpoint de solo-test que exponga el último token generado — **evaluar si conviene un perfil test con un endpoint de bypass de verificación**, dado que Postman no puede leer una casilla de email real. Recomendado: sí, crear un endpoint auxiliar solo activo bajo `spring.profiles.active=test` que devuelva el token sin pasar por email, exclusivamente para esta suite.
6. Login cliente / comercio → 200, guarda tokens.
7. Login admin (usuario admin sembrado por Flyway o por un script de seed) → 200.
8. Admin aprueba el comercio → 200.
9. Admin crea categoría y tag → 201.
10. Comercio crea el producto (sin imágenes todavía) → 201, guarda `producto_id`.
11. Comercio pide firma de Cloudinary para ese producto → 200 (test: la respuesta no debe incluir `api_secret` en ningún campo).
12. Comercio agrega una imagen al producto (con `url` de prueba en el body de `ImagenProductoRequestDTO`, ya que Postman no sube el binario real a Cloudinary salvo que se automatice aparte) → 201.
13. Comercio intenta agregar una sexta imagen tras haber cargado 5 → 409 (test negativo específico del límite de la galería).
14. Cliente ve catálogo → 200, comercio aparece, el producto trae su galería de imágenes.
15. Cliente agrega producto al carrito → 200/201.
16. Cliente intenta agregar producto de otro comercio → 409 (test negativo).
17. Cliente confirma pedido → 201, guarda `pedido_id`.
18. Comercio ve el pedido pendiente → 200.
19. Comercio acepta el pedido → 200.
20. Cliente ve notificación de aceptación → 200.
21. Comercio rechaza un pedido distinto con motivo → 200, cliente recibe notificación de rechazo.
22. Casos negativos: login con password incorrecta (401), acceso a endpoint de admin con token de cliente (403), producto inexistente (404), registro con `localidad_id` inexistente (400/404).

14.5 Seed de datos para testing

Se necesita un **usuario Administrador** precargado (no hay flujo de registro de admin en el MVP, es coherente con el proyecto completo: el admin es un operador interno, no se auto-registra). Esto se resuelve con una migración Flyway adicional (`V11__seed_admin.sql` , después de la carga de geografía y del resto del esquema) que inserte un admin con password ya hasheada en BCrypt, o con un `CommandLineRunner` que lo cree solo si no existe — **recomendado: Flyway**, para que quede versionado igual que el resto del schema. El catálogo `provincia / localidad` ya debe estar cargado

desde la Fase 2bis antes de correr esta suite — sin eso, el registro de cliente/comercio no tiene `localidad_id` válido para usar.

Checklist de cierre de Fase 14

- [] Colección completa exportada a `postman/Bajonea-MVP.postman_collection.json`.
 - [] Environment exportado a `postman/Bajonea-Local.postman_environment.json`.
 - [] Corrida completa con Collection Runner sin fallos en el flujo feliz.
 - [] Casos negativos (401/403/404/409) verificados explícitamente.
 - [] La firma de Cloudinary nunca expone `api_secret` en la respuesta.
 - [] El límite de 5 imágenes por producto se verifica explícitamente (sexta imagen → 409).
-

Fase 15 — Figma: checklist de pantallas MVP

Con el alcance ya cerrado, esta es la lista definitiva de pantallas a diseñar — **no más de esto**, para no volver a tocar las ~100 pantallas del proyecto completo.

15.1 Listado cerrado de pantallas (17 pantallas)

Públicas / sin autenticación:

1. Landing / Catálogo de comercios (listado con filtro básico visual).
2. Detalle de comercio + menú de productos (con filtro por categoría/tag).
3. Login (selector o detección de rol, formulario único de email/password).
4. Registro — Cliente (incluye selector Provincia → Localidad dependiente en el bloque de dirección).
5. Registro — Comercio (idem).
6. Pantalla de "verificá tu email" (post-registro).
7. Pantalla de confirmación de verificación (post-click del link del email).

Cliente autenticado: 8. Carrito. 9. Confirmación de pedido (elegir retiro/domicilio). 10. Mis pedidos (listado + estado). 11. Notificaciones (cliente).

Comercio autenticado: 12. Panel de productos (listado + acciones). 13. Alta/edición de producto (con carga de galería de imágenes vía Cloudinary, hasta 5, marcando cuál es la principal — puede ser modal o pantalla separada). 14. Pedidos recibidos (listado, con acción aceptar/rechazar). 15. Notificaciones (comercio).

Administrador autenticado: 16. Comercios pendientes de aprobación (listado + acción aprobar/rechazar). 17. Gestión de categorías y tags (CRUD simple, puede ser una sola pantalla con dos tabs).

15.2 Qué hacer vos en Figma

Diseñar estas 17 pantallas (podés reutilizar componentes del proyecto completo si ya los tenés en tu librería de Figma — botones, inputs, cards de producto, el selector de dirección — no hace falta empezar de cero). No hace falta conectar las interacciones del prototipo en Figma para este MVP: dado que vas a construir HTML/CSS/JS real en la Fase 16, el prototipo interactivo de Figma es solo para validar el diseño visual antes de codear, no para reemplazar el código.

15.3 Qué pedirle a Claude Code en esta fase

Nada todavía de generación — esta fase es de diseño manual tuyo en Figma. Cuando termines cada pantalla, exportá el frame como imagen y podés pedirle a Claude Code (con el agente de Figma si tenés el MCP conectado, o simplemente adjuntando la imagen) que la traduzca a HTML/CSS en la Fase 16.

Checklist de cierre de Fase 15

- [] 17 pantallas diseñadas en Figma.
- [] El formulario de dirección (registro cliente y comercio) refleja el selector Provincia → Localidad, no un campo de texto libre.
- [] Ningún elemento de flujo fuera de alcance (WhatsApp, MercadoPago, galería de imágenes múltiple, etc.) presente en los diseños.
- [] Diseños exportados como imagen o accesibles vía link para referenciar en la Fase 16.

Fase 16 — Frontend HTML/CSS/JS

16.1 Estructura de carpetas del frontend

```
frontend/
├─ index.html                <- catálogo público
├─ comercio-detalle.html
├─ login.html
├─ registro-cliente.html
├─ registro-comercio.html
├─ verificar-email.html
├─ carrito.html
├─ mis-pedidos.html
├─ notificaciones.html
├─ comercio/
│   └─ productos.html
│   └─ pedidos.html
│   └─ notificaciones.html
├─ admin/
│   └─ comercios-pendientes.html
│   └─ categorias-tags.html
├─ css/
│   └─ styles.css           <- o uno por sección si crece mucho
└─ js/
```

```

|   ├── api.js                <- wrapper fetch con base URL y manejo de JWT
|   ├── auth.js
|   ├── geografia.js          <- selector Provincia → Localidad dependiente
|   ├── catalogo.js
|   ├── carrito.js
|   ├── pedidos.js
|   ├── productos.js
|   ├── cloudinary.js         <- pedir firma al backend + subir directo a Cloudinary
|   ├── admin.js
|   └── notificaciones.js     <- lógica de polling
└── assets/

```

16.2 js/api.js — wrapper centralizado

Un único punto que agrega el header `Authorization: Bearer <token>` (leído de `localStorage`, ya que no hay Web Storage prohibido para esto — es distinto al caso de Artifacts de Claude, esto es una app real corriendo en su propio dominio), maneja errores HTTP de forma centralizada (si recibe `401`, redirige a login), y parsea siempre el `ApiResponse` estándar del backend.

16.3 js/geografia.js — selector dependiente Provincia → Localidad

Al cargar `registro-cliente.html` / `registro-comercio.html`: `GET /api/v1/geografia/provincias` puebla el primer `<select>`. Al elegir una provincia, `GET /api/v1/geografia/localidades?provinciaId=X` puebla el segundo `<select>`, deshabilitado hasta que haya provincia elegida. Este patrón es el mismo que ya tenías previsto en la documentación funcional completa — no es una simplificación, es el flujo real.

16.4 js/cloudinary.js — subida de imágenes de producto (galería, hasta 5)

1. Al guardar un producto con una o más imágenes nuevas: por cada imagen, `POST /api/v1/productos/{id}/cloudinary/firma` (autenticado) → recibe `CloudinarySignatureResponseDTO`. Si el producto ya llegó a 5 imágenes, este paso devuelve `409` y el frontend debe deshabilitar el botón de "agregar imagen" en la UI antes de intentarlo.
2. Con esos datos, hace un `POST` directo a `https://api.cloudinary.com/v1_1/{cloudName}/image/upload` (multipart, sin pasar por el backend) por cada archivo.
3. Toma la `secure_url` de cada respuesta de Cloudinary y arma la `List<ImagenProductoRequestDTO>` (url, orden, esPrincipal) que se manda al backend junto con el resto del producto.
4. En la pantalla de edición, permite además operaciones sueltas: agregar una imagen a un producto ya existente, eliminar una imagen puntual de la galería, y marcar cuál es la principal — sin tener que reenviar el producto completo.

16.5 Flujo de implementación recomendado (una pantalla a la vez, no todo junto)

1. `login.html` + `js/auth.js` — primero esto, porque todo lo demás depende de tener un token.
2. `registro-cliente.html` / `registro-comercio.html` + `js/geografia.js` (el selector de dirección es parte central de ambos formularios).
3. `verificar-email.html` .
4. `index.html` (catálogo) — es pública, buen paso para validar que el fetch básico funciona sin JWT todavía.
5. `comercio-detalle.html` con filtro de categoría/tag.
6. `carrito.html` .
7. `mis-pedidos.html` (cliente) + confirmación de pedido.
8. `comercio/productos.html` (CRUD) + `js/cloudinary.js` para la imagen.
9. `comercio/pedidos.html` (aceptar/rechazar).
10. `admin/comercios-pendientes.html` .
11. `admin/categorias-tags.html` .
12. `notificaciones.html` + polling — al final, porque es transversal y se prueba mejor cuando ya hay eventos reales que la disparan (pedidos, aprobaciones).

16.6 Diseño visual

Usar la skill `frontend-design` (ya disponible) al momento de escribir el CSS, para que no termine con apariencia de plantilla genérica de Bootstrap por defecto — coherente con que este es un proyecto real con identidad propia ("Bajoneá"), no un prototipo descartable.

Checklist de cierre de Fase 16

- [] Las 17 pantallas de Figma tienen su equivalente HTML/CSS/JS funcional.
 - [] Cada pantalla consume el backend real (no datos mockeados) en su versión final.
 - [] El selector Provincia → Localidad funciona en ambos formularios de registro.
 - [] La subida de imágenes a Cloudinary funciona de punta a punta (hasta 5 por producto) y las URLs persisten correctamente en la galería del producto, con una marcada como principal.
 - [] El flujo completo (registro → verificación → login → catálogo → carrito → pedido → aceptación → notificación) se puede recorrer manualmente de punta a punta en el navegador.
-

Fase 17 — Testing E2E con Playwright

17.1 Instalación

```
npm init playwright@latest
```

Dentro de `testing/playwright/` . Elegir TypeScript o JavaScript (recomendado TypeScript, dado que ya trabajás con tipado fuerte en el backend — mantiene consistencia de criterio, aunque no es obligatorio).

17.2 Estructura de specs (calcada al orden de la Fase 16)

```
testing/playwright/
├─ playwright.config.ts
├─ tests/
│   ├─ 01-registro-y-verificacion.spec.ts
│   ├─ 02-login.spec.ts
│   ├─ 03-catalogo-publico.spec.ts
│   ├─ 04-carrito.spec.ts
│   ├─ 05-pedido-flujo-completo.spec.ts    <- el más importante: cubre todo el circuito cliente
│   ├─ 06-crud-productos.spec.ts
│   ├─ 07-crud-categorias-tags.spec.ts
│   ├─ 08-aprobacion-comercio.spec.ts
│   └─ 09-notificaciones.spec.ts
```

17.3 El test más importante: 05-pedido-flujo-completo.spec.ts

Este es el que valida el corazón funcional del MVP tal como lo definiste: simular un pedido de punta a punta.

1. Login como cliente (usuario ya verificado, seedeado).
2. Ir al catálogo, entrar a un comercio aprobado.
3. Agregar un producto al carrito.
4. Confirmar el pedido (retiro).
5. Assert: aparece en "Mis pedidos" con estado PENDIENTE.
6. [Nuevo contexto de browser] Login como el comercio dueño del producto.
7. Assert: el pedido aparece en "Pedidos recibidos" con estado PENDIENTE.
8. Assert: hay una notificación nueva para el comercio.
9. Aceptar el pedido.
10. [Volver al contexto del cliente] Refrescar "Mis pedidos".
11. Assert: el estado cambió a ACEPTADO.
12. Assert: hay una notificación nueva para el cliente.

Playwright soporta múltiples `browserContext` en un mismo test, lo cual permite simular cliente y comercio como dos sesiones separadas dentro del mismo spec — ideal para este caso.

17.4 Datos de prueba

Igual que en Postman (13.5), Playwright necesita un backend en un estado conocido antes de correr. Opciones: (a) un script `beforeAll` que llame a los mismos endpoints de `seed/reset` vía `request` de Playwright (no solo UI), o (b) una base de datos de test separada (`bajonea_test`) que se resetea con Flyway `clean + migrate` antes de la suite. **Recomendado para un TFC: opción (b)**, más prolijo de defender y evita contaminar datos de la demo con datos de test.

17.5 Selectores

Usar `data-testid` en el HTML de la Fase 16 para los elementos clave (`data-testid="btn-agregar-carrito"` , `data-testid="estado-pedido"`) en vez de depender de clases CSS que puedan cambiar con el diseño — esto evita que un ajuste visual rompa los tests.

Checklist de cierre de Fase 17

- [] Los 9 specs corren en verde contra el entorno local completo (backend + frontend + MySQL).
 - [] El spec 05 (flujo de pedido) pasa de forma consistente, no flaky (si falla intermitentemente, suele ser por falta de `await` en el polling de notificaciones — usar `expect.poll()` o `page.waitForResponse()` en vez de esperas fijas).
 - [] Reporte HTML de Playwright (`npm playwright show-report`) revisado al menos una vez.
-

Fase 18 — Integración final y checklist de cierre del MVP

18.1 Checklist funcional (contra la imagen adjunta y el alcance definido)

- [] Editar perfil del usuario (cliente).
- [] Filtrar categorías de alimentos (en el menú de un comercio).
- [] Carrito de compras (cliente).
- [] Pedido simulado (reemplaza "conectar por WhatsApp" según lo acordado): POST exitoso, notifica al comercio.
- [] Logo y nombre de la aplicación en el frontend ("Bajoneá").
- [] Cambio de clave — **nota:** la imagen menciona "recuperación por correo (página de recuperación)"; esto no fue parte de las preguntas de alcance iniciales. Confirmar con vos si entra al MVP o queda para la siguiente iteración (recomendación: dado que ya se resuelve la infraestructura SMTP en la Fase 10 para verificación, el costo marginal de sumar "recuperar contraseña" con el mismo mecanismo de `Token` es bajo — **evaluar sumarlo como Fase 10bis opcional**).
- [] Notificar nuevo pedido (comercio) — cubierto en Fase 12.
- [] Solicitud de comercios: aprobar o rechazar con motivo — cubierto en Fase 8.3.
- [] Selector Provincia/Localidad en el registro — cubierto en Fase 2bis + Fase 16.3.
- [] Imagen de producto vía Cloudinary (galería de hasta 5, con principal) — cubierto en Fase 11.
- [] Desplegar la app en un servidor — ver 18.3.

18.2 Checklist técnico general

- [] Todos los endpoints devuelven `ApiResponse<DTO>` , nunca entidades.
- [] Todos los status HTTP son semánticamente correctos.
- [] JWT funcionando con los 3 roles y las reglas de autorización correctas.
- [] Colección Postman completa y en verde.
- [] Suite Playwright completa y en verde.
- [] Sin credenciales hardcodeadas en el repo.

- `[] CLAUDE.md` y `docs/DECISIONES.md` actualizados con las decisiones tomadas durante la implementación (no solo las de este documento).

18.3 Despliegue (mencionado en la imagen, fuera del detalle línea por línea de esta guía)

Esto excede el alcance de "guía de implementación de funcionalidades" y entra en infraestructura — ya tenés documentado el stack de hosting (DonWeb VPS, Nginx, systemd, Cloudflare) en `stack-tecnologico.md`. Se recomienda tratarlo como una **guía separada** una vez que el MVP esté funcionalmente cerrado y testeado localmente, para no mezclar dos tipos de trabajo muy distintos en un mismo documento. Si querés, puedo armar esa guía de despliegue como documento aparte cuando lleguemos a esa etapa.

Anexo A — Skills a crear (detalle completo)

Recordatorio del PDF: la descripción de la skill es el mecanismo de disparo — tiene que ser específica y "empujadora", nombrando contextos aunque no se use la palabra exacta.

A.1 `skill-generar-capacrud`

Propósito: generar Entity + DTOs (request/response) + Repository + Service + Controller para un recurso nuevo, siguiendo exactamente las convenciones de las Fases 3 a 9 (paquetes, `ApiResponse`, status HTTP, `List` por defecto, sin exponer entidades).

Cuándo se dispara: cualquier pedido de "agregá el CRUD de X", "necesito un nuevo recurso Y", o al reconocer un patrón de "entidad + repositorio + service + endpoint" en la conversación, incluso si no se usan esas palabras exactas.

Contenido del SKILL.md:

- Plantilla de cada archivo (Entity, DTOs, Repository, Service, Controller) con placeholders.
- Checklist de validación post-generación (¿el controller devuelve `ApiResponse`? ¿el status es el correcto según el verbo HTTP? ¿la entidad tiene `@EqualsAndHashCode(of="id")`?).
- Referencia a `docs/modelo-mvp.md` como fuente de verdad de columnas/relaciones.

A.2 `skill-generar-test-postman`

Propósito: dado un controller ya implementado, generar el bloque de requests + tests de Postman correspondiente (formato JSON de colección o snippet para pegar), incluyendo casos positivos y al menos un caso negativo (401/403/404/409 según aplique).

Cuándo se dispara: "agregá los tests de Postman para X", "necesito probar este endpoint", o automáticamente al cerrar una Fase de controllers nueva.

A.3 skill-generar-spec-playwright

Propósito: generar un spec de Playwright siguiendo el patrón de la Fase 17 (multi-contexto cuando aplica cliente+comercio, uso de `data-testid`, `expect.poll()` para casos de polling).

Cuándo se dispara: "agregá el test E2E de X", "probemos esta pantalla con Playwright".

A.4 skill-auditoria-cruzada

Propósito: replicar tu práctica ya establecida en la documentación (cross-validar todos los artefactos antes de avanzar de fase). Dada una fase recién cerrada, revisa que Entity ↔ Migración Flyway ↔ DTO ↔ Repository ↔ Service ↔ Controller ↔ (si aplica) request de Postman estén alineados, y reporta **todas** las inconsistencias encontradas en un solo pase — no iterativamente, según tu preferencia ya registrada.

Cuándo se dispara: al cierre de cada Fase (4 a 9 especialmente), o explícitamente "audité lo que llevamos hasta ahora".

Anexo B — Agentes a crear (detalle completo)

Según el PDF: un agente es para trabajo acotado y delegable al 100%, que además conviene aislar del contexto principal (por ejemplo, para no ensuciar la sesión principal con la lectura de decenas de archivos).

B.1 agente-auditor-consistencia

Herramientas permitidas: solo lectura (leer archivos, buscar en el repo). Sin escritura.

Tarea: recorrer todo `backend/src` más `docs/modelo-mvp.md` y devolver un reporte único de inconsistencias (columna en Flyway sin campo en Entity, DTO con campo que no existe en la Entity, Service que no valida algo que el Repository permite, etc.). Reporta agrupado por severidad (crítico/alto/medio/bajo), con archivo y línea.

Por qué agente y no skill: esto requiere leer muchos archivos y el resultado que le interesa a la sesión principal es solo el resumen final, no el proceso de lectura completo — coherente con el punto 5.2 del PDF ("cuidar el contexto del proceso principal").

B.2 agente-generador-coleccion-postman

Herramientas permitidas: lectura de `backend/src/main/java/.../controllers/**`, escritura únicamente dentro de `postman/`.

Tarea: dado el estado actual de todos los controllers, regenerar o actualizar la colección Postman completa (`Bajonea-MVP.postman_collection.json`) manteniendo la organización por carpetas de la

Fase 13, sin duplicar requests ya existentes.

Por qué agente y no skill: el trabajo es de "barrido completo del proyecto para producir un artefacto derivado" — más pesado y menos frecuente que generar un CRUD puntual (que sí es una skill).

Anexo C — Hooks a crear (detalle completo)

Según el PDF: si algo debe cumplirse siempre, sin excepción, y sin depender del criterio del modelo bajo presión de contexto, es un hook — no una instrucción más enfática en el prompt.

C.1 Hook `pre-tool-use` — bloquear controllers que devuelven Entity

Evento: después de escribir/editar un archivo en `controllers/` (post-tool-use sobre `str_replace / create_file`).

Lógica determinística: grep sobre el archivo modificado buscando firmas de método `@GetMapping / @PostMapping /etc.` cuyo tipo de retorno no sea `ResponseEntity<ApiResponse<...>>` (o `ResponseEntity<ApiResponse<List<...>>>`). Si encuentra una coincidencia con un tipo que matchea un nombre de clase en `entities/` , bloquea y devuelve un mensaje claro: "Este controller devuelve una entidad directamente. Envolve el resultado en un DTO + ApiResponse antes de continuar."

C.2 Hook `pre-tool-use` — bloquear escritura fuera de las carpetas del proyecto

Evento: antes de cualquier escritura de archivo.

Lógica: rechazar cualquier ruta que no esté bajo `backend/` , `frontend/` , `postman/` , `testing/` , `docs/` , o `.claude/` . Evita que una sesión larga termine escribiendo algo en un lugar inesperado del sistema de archivos.

C.3 Hook `post-tool-use` — formateo automático

Evento: después de cada edición de un archivo `.java` .

Lógica: correr el formateador configurado (ej. `google-java-format` o el que ya tengas integrado en tu IDE/Maven plugin) automáticamente, sin pasar por decisión del modelo.

C.4 Hook `post-tool-use` — bloquear `set / tuplas` sin justificación

Evento: después de editar un archivo en `services/` o `repositories/` .

Lógica: si aparece `Set<` o `Pair</ Tuple` en el diff y no hay un comentario `// justificacion:` en la misma línea o la anterior, advertir (no necesariamente bloquear — esto es más una convención de estilo que una regla de seguridad, así que puede implementarse como advertencia no bloqueante si preferís no ser demasiado estricto).

Anexo D — Estructura de carpetas final del repo (referencia)

```
bajonea/
├─ CLAUDE.md
├─ docs/
│   └─ DECISIONES.md
│   └─ modelo-mvp.md
├─ .claude/
│   └─ skills/
│       └─ generar-capacrud/SKILL.md
│       └─ generar-test-postman/SKILL.md
│       └─ generar-spec-playwright/SKILL.md
│       └─ auditoria-cruzada/SKILL.md
│   └─ agents/
│       └─ auditor-consistencia.md
│       └─ generador-coleccion-postman.md
│   └─ hooks/
│       └─ bloquear-entity-en-controller.*
│       └─ bloquear-escritura-fuera-de-proyecto.*
│       └─ formateo-automatico.*
│       └─ advertir-set-sin-justificacion.*
├─ backend/
│   └─ pom.xml
│   └─ src/main/java/com/bajonea/backend/
│       └─ BajoneaApplication.java
│       └─ config/                <- incluye CloudinaryConfig
│       └─ dto/
│       └─ entities/              <- incluye Provincia, Localidad
│       └─ repositories/
│       └─ services/              <- incluye GeografiaService, CloudinaryService
│       └─ controllers/           <- incluye GeografiaController
│       └─ exceptions/
│   └─ scripts/etl-georef/        <- carga inicial del catálogo geográfico (Fase 2bis)
│   └─ src/main/resources/
│       └─ application.properties
│       └─ db/migration/
│           └─ V1__geografia.sql
│           └─ V2__usuarios_y_personas.sql
│           └─ V3__roles.sql
│           └─ V4__direcciones.sql
│           └─ V5__tokens.sql
│           └─ V6__categorias_y_tags.sql
│           └─ V7__productos.sql
│           └─ V8__carrito.sql
│           └─ V9__pedidos.sql
│           └─ V10__notificaciones.sql
│           └─ V11__seed_admin.sql
├─ frontend/
│   └─ *.html
│   └─ css/
│   └─ js/
│   └─ admin/ , comercio/
```

```
└─ postman/
   └─ Bajonea-MVP.postman_collection.json
└─ Bajonea-Local.postman_environment.json
└─ testing/
   └─ playwright/
      └─ playwright.config.ts
      └─ tests/
```

Cómo seguir a partir de acá

El orden recomendado de ejecución con Claude Code es literalmente el de las Fases 0 a 18 (con la Fase 2bis intercalada justo después de la Fase 2). Dentro de cada fase, no pasar a la siguiente sin cerrar el checklist correspondiente — es la misma disciplina de "cross-validar antes de avanzar" que ya venís aplicando en toda la documentación previa, ahora aplicada al código.

Cuando arranques la Fase 0, decime y avanzamos definiendo el `CLAUDE.md` inicial juntos.